# EduTrace

## Project Progress Report and SRS-Style Architecture Document

*Current prototype built with Flask, SQLite, sessions, and filesystem certificate storage*

Prepared for: Hackathon / Prototype submission

Focus: Backend implementation, data flow, and feature summary

Status: Working prototype with multi-role authentication and certificate upload pipeline

*Generated from the current EduTrace implementation*

## Contents



## 1. Project Overview

EduTrace is a role-based certificate management prototype. The system currently supports an admin, authorities, and students. The admin creates authorities, authorities add students, and authorities upload PDF certificates for students. Certificate files are stored on disk, while certificate metadata is stored in SQLite.

The long-term goal of the project is to extend this structure into a certificate verification and blockchain-backed immutability system. For now, the prototype focuses on the complete working backend pipeline.

## 2. Current System Architecture

The current application is built in Flask. The homepage routes users to role-specific login pages. After login, the user enters a dashboard for that role. Sessions are used to remember the logged-in authority so that student onboarding and certificate upload can be tied to the correct authority.

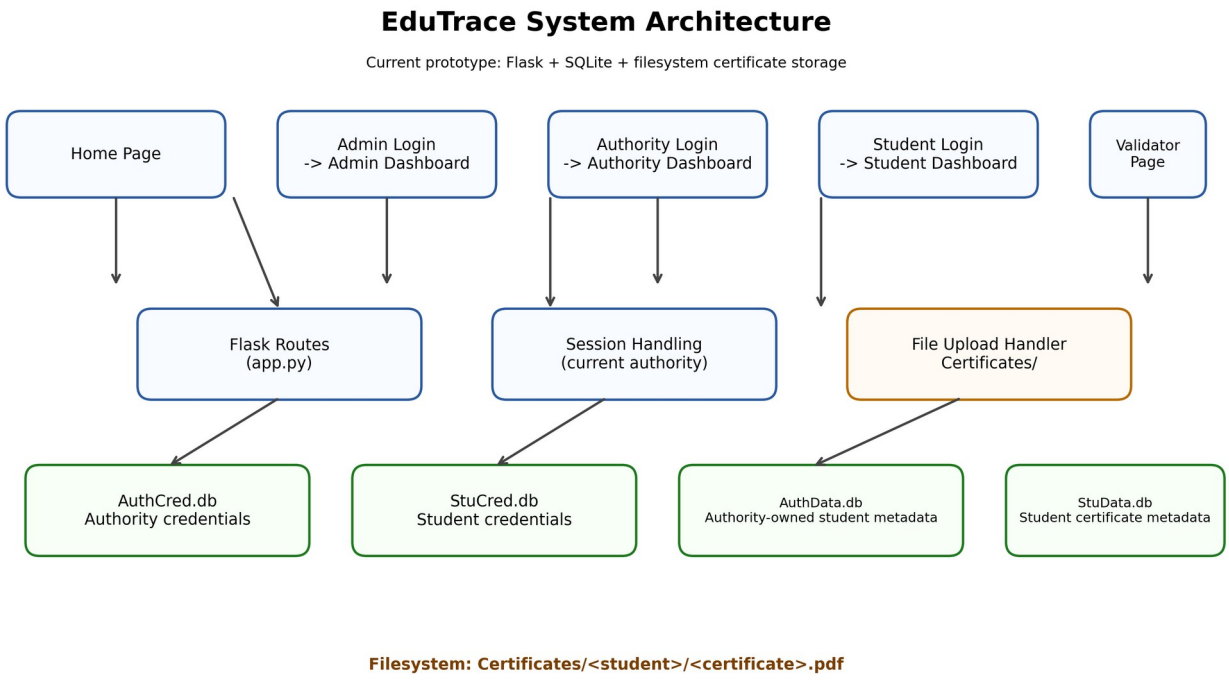


*Figure 1: EduTrace system architecture and storage layout.*

## 3. Flow of Data Through the System

The data flow is intentionally layered. Each action performed in the user interface reaches a Flask route, which then updates the relevant database or filesystem location.

High-level flow:

1. User opens the homepage and selects a role.
2. Flask loads the corresponding login page.
3. Credentials are checked against the SQLite database for that role.
4. If login succeeds, the dashboard is shown.
5. The logged-in authority is stored in session memory.
6. That authority can add students and upload certificates.
7. Certificate files are saved under a dedicated student folder.
8. Certificate metadata is stored in StuData.db.

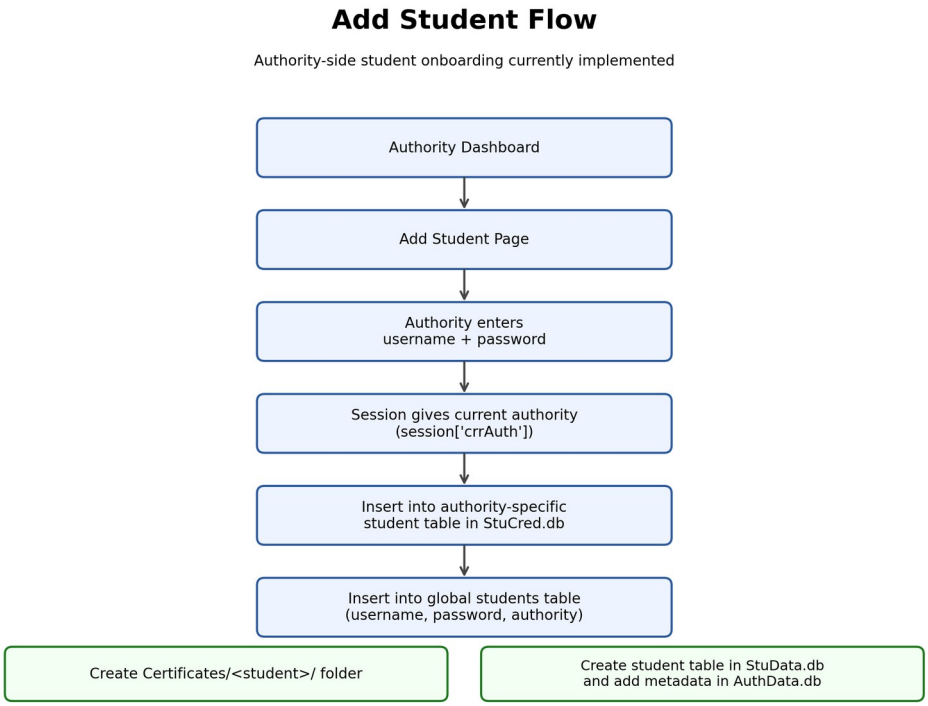


*Figure 2: Flow of data when an authority adds a student.*

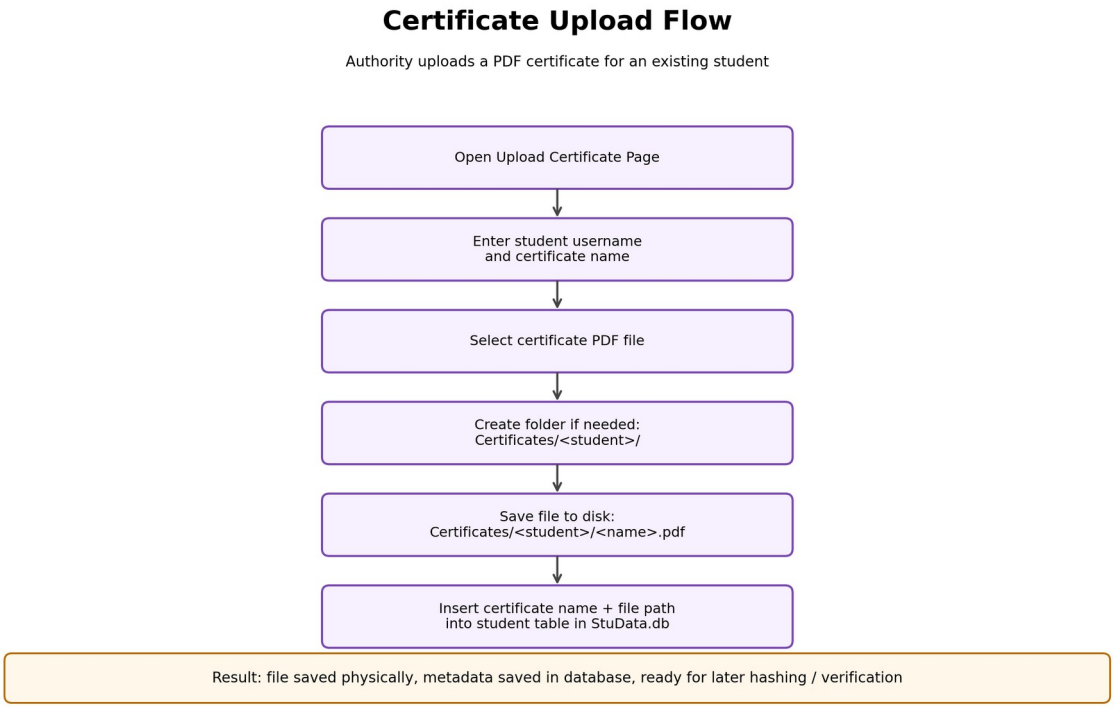


*Figure 3: Flow of data when a certificate is uploaded.*

## 4. Feature Summary by Role

The following features have already been implemented in the prototype.

### 4.1 Admin Features

- Admin login page and dashboard.
- Admin credential verification from AdminCred.db.
- Add authority functionality.
- Authority account creation in AuthCred.db.
- Automatic creation of authority-owned student tables and metadata tables.

### 4.2 Authority Features

- Authority login using session-based authentication.
- Add student functionality.
- Automatic creation of a student directory in Certificates/.
- Student insertion into authority-specific and global student tables.
- Certificate upload page.
- Upload of PDF certificates for a specific student.
- Saving the uploaded file to a dedicated folder and saving metadata in StuData.db.

### 4.3 Student Features

- Student login page and dashboard.
- Student credential verification from StuCred.db.
- Student role is prepared for future certificate viewing and download.

## 5. Database and Filesystem Design

The prototype uses SQLite for structured data and the filesystem for physical certificate storage. This keeps the system fast and easy to manage while still preserving proper separation between data and files.

| Storage | Purpose | Examples |
|---|---|---|
| AdminCred.db | Admin credentials | admins table |
| AuthCred.db | Authority credentials | authorities table |
| StuCred.db | Student credentials and authority-owned student tables | students table + one table per authority |
| AuthData.db | Authority-owned student metadata | one table per authority |
| StuData.db | Student certificate metadata | one table per student |
| Certificates/ | Physical PDF storage | Certificates/<student>/ <certificate>.pdf |

The certificate itself is not stored inside the database. Instead, the system saves the PDF on disk and stores the certificate name and file path in the student's table. This is the correct prototype design because it keeps the database lightweight and the file handling simple.

## 6. Implemented Certificate Upload Pipeline

The upload pipeline currently behaves like this:

9. Authority opens the Upload Certificate page.
10. Authority enters the student username and certificate name.
11. Authority selects a PDF file.
12. Flask receives the file in request.files.
13. A student folder is created if it does not already exist.
14. The PDF is saved to Certificates/<student>/<certificate>.pdf.
15. StuData.db is updated with certificate name and path.
16. A success message is returned to the dashboard.

This feature is one of the most important parts of the project because it proves that the system can move data from the web page into the filesystem and then into the database in a controlled way.

## 7. Current Progress and Next Steps

What is already working:

- Multi-role login system.
- SQLite-based credential storage.
- Session-based authority tracking.
- Admin-to-authority onboarding.
- Authority-to-student onboarding.
- Certificate file upload and metadata storage.

What comes next:

- Certificate verification layer.
- Hash generation for uploaded documents.
- Validator dashboard.
- Blockchain metadata storage for tamper detection.
- Student certificate viewing and download.

## 8. Conclusion

EduTrace has reached a strong prototype stage. The backend is now capable of handling role-based access, dynamic student onboarding, file upload, and metadata persistence. The next stage can focus on verification and blockchain integration without changing the basic system structure.